

Ayush Kumar (230261)
EE604 Image Processing
Assignment 1

Question 1:

Input Grayscale Image:



Code->

```
img = cv2.imread("input/img", cv2.IMREAD_GRAYSCALE)    # Read the image as a
                                                         grayscale image

print("Shape:", img.shape)    # height and width of the image

print("Data type:", img.dtype)    # data type of pixel values

final_img = np.zeros_like(img, dtype=np.uint8)    # Empty matrix with the same size as
the original image

stage = 1 # Initialize the stage number
```

```

frames = []      # Empty List to store the frame

for b in range(7, -1, -1):    # Loop through bits from 8th bit to 1st bit

    b_p = (img >> b) & 1      # Extract the current bit from every pixel

    final_img += b_p * (2 ** b)    # Add the extracted bit to reconstruct the image

frames.append(final_img.copy())    # Store a copy of the current reconstructed image
                                   # as a frame


height, width = img.shape    # Get the height and width of the image

fourcc = cv2.VideoWriter_fourcc(*'mp4v')    # Define the codec used to create the MP4
video                                         # video

video = cv2.VideoWriter('Final_image.mp4', fourcc, 1, (width, height), False)    #
Create a video with 1 frame per second and grayscale frames

for frame in frames:    # each frame

    video.write(frame)    # Add the current frame

video.release()

#Code For Generating All the 8 Frame in One Image

reconstruct = np.zeros_like(img, dtype=np.uint8)

plt.figure(figsize=(12, 8))

stage = 1

for bit in range(7, -1, -1):

    bit_plane = (img >> bit) & 1

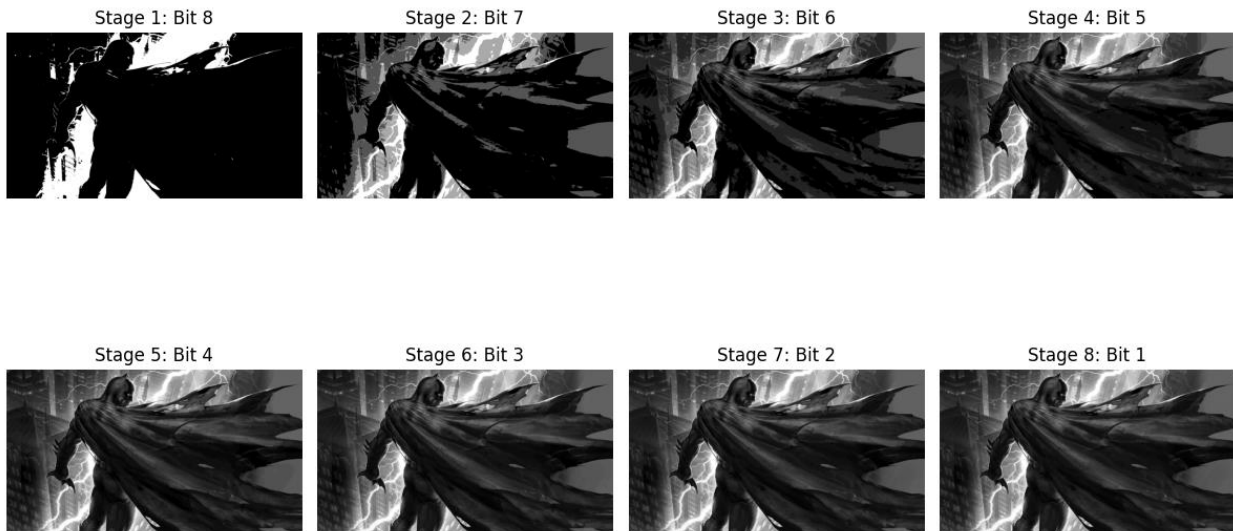
    reconstruct += bit_plane * (2 ** bit)

plt.subplot(2, 4, stage)

```

```
plt.imshow(reconstruct, cmap='gray')  
  
plt.title(f"Stage {stage}: Bit {bit + 1}")  
  
plt.axis('off')  
  
stage += 1  
  
plt.tight_layout()  
  
plt.show()
```

All the 8 Frames :



Question 2;

Input Grayscale Image :

Original Clean Grayscale Image



Filters used :

1. *# Gaussian Noise → Arithmetic Mean Filter*

Original Image



Gaussian Noise



Arithmetic Mean Filter



PSNR : 23.2844 dB SSIM : 0.5993

Gaussian noise with mean = 0 and sigma = 25

```
np.random.seed(42)
noise = np.random.normal(0, 25, original.shape)

# Noise Addition
noisy_image = original + noise
noisy_image = np.clip(noisy_image, 0, 255)

# filter 3 × 3
k = 3
pad = k // 2

# Added padding
padded = np.pad(noisy_image, pad, mode="edge")

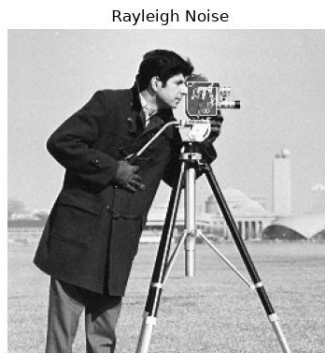
# Empty Image
restored_image = np.zeros_like(noisy_image)

for i in range(noisy_image.shape[0]):
    for j in range(noisy_image.shape[1]):
        # Extract the 3 × 3 neighbourhood
        window = padded[i:i+k, j:j+k]

        # Replace pixel with the neighbourhood mean
        restored_image[i, j] = np.mean(window)

        # Display images and calculate scores
show_result(noisy_image, restored_image, "Gaussian Noise", "Arithmetic Mean")
```

2. *# Rayleigh Noise → Geometric Mean Filter*



PSNR : 23.8816 dB SSIM : 0.8116

random values between 0 and 1

```
np.random.seed(42)
```

```
u = np.random.uniform(0, 1, original.shape)
```

Rayleigh noise

```
noise = np.sqrt(-2 * 20 * np.log(1 - u))
```

Removed average shift from the noise

```
noise = noise - np.mean(noise)
```

Add noise to the original image

```
noisy_image = original + noise
```

pixel values between 0 and 255

```
noisy_image = np.clip(noisy_image, 0, 255)
```

filter size to 3×3

```
k = 3
```

```
pad = k // 2
```

padding around the image

```
padded = np.pad(noisy_image, pad, mode="edge")
```

empty restored image

```
restored_image = np.zeros_like(noisy_image)
```

```
for i in range(noisy_image.shape[0]):
```

```
for j in range(noisy_image.shape[1]):
```

```
# Extract 3 × 3 neighbourhood
```

```
window = padded[i:i+k, j:j+k]
```

```
# Avoid log(0) while calculating geometric mean
```

```
window = window + 1e-8
```

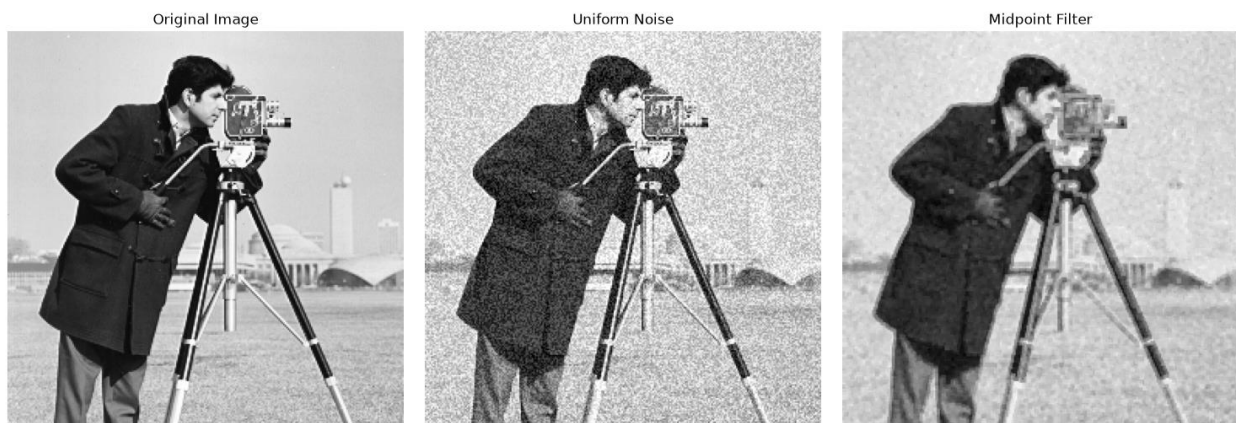
```
# Calculated geometric mean using logarithms
```

```
restored_image[i, j] = np.exp(np.mean(np.log(window)))
```

```
# Display images and calculate scores
```

```
show_result(noisy_image, restored_image, "Rayleigh Noise", "Geometric Mean")
```

```
3. # Uniform Noise → Midpoint Filter
```



PSNR :21.5566 dB SSIM :0.6275

```
# uniform noise between -40 and +40
```

```
np.random.seed(42)
```

```
noise = np.random.uniform(-40, 40, original.shape)
```

```
# noise to the original image
```

```
noisy_image = original + noise
```

```
# pixel values between 0 and 255
```

```

noisy_image = np.clip(noisy_image, 0, 255)

# filter size to 3 × 3

k = 3

pad = k // 2

# padding around the image
padded = np.pad(noisy_image, pad, mode="edge")

# empty restored image
restored_image = np.zeros_like(noisy_image)

for i in range(noisy_image.shape[0]):
    for j in range(noisy_image.shape[1]):
        # 3 × 3 neighbourhood
        window = padded[i:i+k, j:j+k]

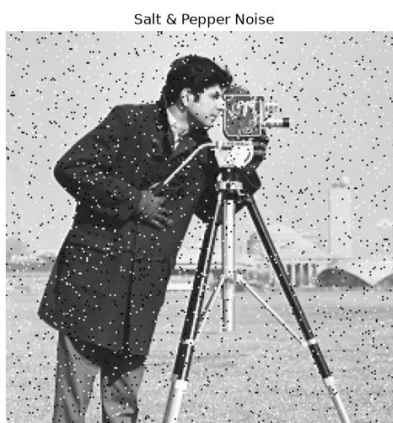
        # minimum and maximum values in the window
        minimum = np.min(window)
        maximum = np.max(window)

        # Midpoint is the average of minimum and maximum
        restored_image[i, j] = (minimum + maximum) / 2

# Display images and calculate scores
show_result(noisy_image, restored_image, "Uniform Noise", "Midpoint")

4. # Salt & Pepper Noise → Median Filter

```



PSNR: 25.7430 dB SSIM: 0.8478

```
np.random.seed(42)
```

```
noisy_image = original.copy()
```

```
# random values for every pixel
```

```
random_values = np.random.random(original.shape)
```

```
# salt noise by setting pixels to 255
```

```
noisy_image[random_values < 0.02] = 255
```

```
# pepper noise by setting pixels to 0
```

```
noisy_image[
```

```
(random_values >= 0.02) &
```

```
(random_values < 0.04)
```

```
] = 0
```

```
# filter size to 3 × 3
```

```
k = 3
```

```
pad = k // 2
```

```
# padding around the image
```

```
padded = np.pad(noisy_image, pad, mode="edge")
```

```
# Empty restored image
```

```
restored_image = np.zeros_like(noisy_image)
```

```
# Visit every pixel
```

```
for i in range(noisy_image.shape[0]):
```

```
for j in range(noisy_image.shape[1]):
```

Extract 3 × 3 neighbourhood

```
window = padded[i:i+k, j:j+k]
```

Replace the pixel with the median value

```
restored_image[i, j] = np.median(window)
```

Display images and calculate scores

```
show_result(noisy_image, restored_image, "Salt & Pepper Noise", "Median")
```

5. *#display ,psnr and ssim value calculation*

```
def show_result(noisy, restored, noise_name, filter_name):
```

```
    psnr = calculate_psnr(original, restored)
```

```
    ssim_value = ssim(original.astype(np.uint8),
```

```
    restored.astype(np.uint8),
```

```
    data_range=255)
```

```
    plt.figure(figsize=(15, 5))
```

```
    plt.subplot(1, 3, 1)
```

```
    plt.imshow(original, cmap="gray")
```

```
    plt.title("Original")
```

```
    plt.axis("off")
```

```
    plt.subplot(1, 3, 2)
```

```
    plt.imshow(noisy, cmap="gray")
```

```
    plt.title(noise_name)
```

```
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
```

```
plt.imshow(restored, cmap="gray")
```

```
plt.title(filter_name)
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print(f"{noise_name} → {filter_name}")
```

```
print(f"PSNR : {psnr:.4f} dB")
```

```
print(f"SSIM : {ssim_value:.4f}")
```